PalmSource™ 99

Problem-Solving Insights: Info in a Handle

Shannon Pekary
President, Hands High Software

3Com
More connected.

Problem Statement

- Problem: How to create a general purpose way to manage information in a handle
  - I found I was messing with handle memory all the time
  - This kind of code was producing a lot of bugs

Considerations

- A handle may be in dynamic RAM or storage RAM
- A storage RAM handle may be tied to a database record, or used as an AppInfo block, or it might not be attached to anything
- The operating system might be < OS 3.0, which means storage handles might be reallocated if they are grown

Basic Handle Manipulator

- MemUtilMunge
- Purpose:
  - General manipulation of handle based memory—this can be dynamic or storage memory
- Preconditions:
  - The combination of offset, len and srcLen cannot make a handle of zero size

Basic Handle Manipulator:

- Input:
  - hDest: handle to change
  - offset: offset into handle to change
  - len: length of data in the handle to change—use 0 for an insert
  - pSrc: pointer to memory to insert—use NULL to simply insert zeros
  - srcLen: length of data to insert—use 0 to delete
  - dbRef: An open database—It's only used to figure out what card to create new database memory

Basic Handle Manipulator

- Output:
  - err: if the memory could not grow, then an error is returned and the handle remains unchanged
- Returns:
  - handle: the handle with the changed memory
  - If you pass storage based memory and the memory had to be reallocated, this will be a different handle than the one you passed, but the old handle WILL STILL BE ALLOCATED, so you have to detect this and dispose of the old one yourself

Basic Handle Manipulator

PalmSource 99

- Pseudo Code
  - If we have to grow memory:
    - Try to resize it
    - If a resize fails, try to reallocate memory somewhere else and copy the old info to the new handle
    - If this fails too, we bail
  - Move old data out of the way if we are doing an insert
  - Copy new data to the right place
  - Shrink the handle if needed

Basic Handle Manipulator

PalmSource 99

- Code

```
VoidHand MemUtilMunge(VoidHand hDest,
                                    Long offset,
                                    Long len,
                                    const void*   pSrc,
                                    Long srcLen,
                                    Err *err,
                                    DmOpenRef dbRef)
{
    Long       newSize = 0;
    Long       oldSize = 0;
    CharPtr    p;
    VoidHand   hNew;
    CharPtr            pOld, pNew;
    Boolean dataStore;
    Long       moveAmount;
```

Basic Handle Manipulator

PalmSource 99

- Code

```
ASSERT (! (srcLen > len && MemHandleLockCount
(hDest)));
ASSERT (offset + len <= MemHandleSize (hDest));

dataStore = MemHandleDataStorage (hDest);
*err = noErr;

if (srcLen != len) {
    // skip A-trap hit if not needed
    oldSize = MemHandleSize (hDest);
    newSize = oldSize - len + srcLen;
}
```

Basic Handle Manipulator

- Code

```
// grow the handle if we need to
if (srcLen > len) {
    ASSERT (newSize != 0); // can't make a zero sized
handle!
    *err = MemHandleResize (hDest, newSize);
    if (*err) {
            if (!dataStore) {
                    return hDest;
            } else {
                    hNew = DmNewHandle (dbRef, newSize);
                    if (hNew == NULL) {
                            *err = dmErrMemError;
                            return hDest;
                    }
```

Basic Handle Manipulator

PalmSource 99

- Code

```
                    // now copy old stuff to new handle
                    pNew = (Char*)MemHandleLock (hNew);
                    pOld = (Char*)MemHandleLock (hDest);
                    DmWrite (pNew, 0, pOld, oldSize);
                    MemPtrUnlock (pNew);
                    MemPtrUnlock (pOld);
                    // don't dispose of hDest here
                    hDest = hNew;
                }
            }
        }
```

Basic Handle Manipulator

PalmSource 99

- Code

```
p = MemHandleLock (hDest);

// move old memory out of the way
if (srcLen != len) {
    // don't take A-trap speed hit if not necessary
    moveAmount = oldSize - offset - len;
    if (dataStore) {
            DmWrite (p, offset + srcLen, p + offset +
                    len, moveAmount);
    } else {
            MemMove (p + offset + srcLen, p + offset +
                    len, moveAmount);
    }
}
```

Basic Handle Manipulator

PalmSource 99

- Code

```
// copy new memory
if (pSrc) {
    if (srcLen) {
            if (dataStore) {
                    DmWrite (p, offset, pSrc, srcLen);
            } else {
                    MemMove (p + offset, pSrc, srcLen);
            }
    }
} else if (srcLen) { // fill with zeros
    if (dataStore) {
            DmSet (p, offset, srcLen, 0);
    } else {
            MemSet (p + offset, srcLen, 0);
    }
}
```

Basic Handle Manipulator

- Code

```
    MemHandleUnlock (hDest);
    if (srcLen < len) {
        ASSERT (newSize != 0);
        MemHandleResize (hDest, newSize); // in case it got
                                          smaller
    }

    return hDest;
} /* MemUtilMunge */
```

Using MemUtilMunge

- Scenarios
  - Dynamic memory
    - Just use it, the returned handle will always be the same, since the dynamic memory heap is one heap
  - Storage memory
    - Great for temporary memory, or stuff that might be bigger than dynamic memory—watch out, because the handle might change
  - Record Memory
    - If the handle changes, you have to reattach the handle to the record
  - AppInfo Memory
    - If the handle changes, you have to reattach it to the AppInfoID part of the database header

Record munge Routine Using MemUtilMunge

PalmSource 99

```
Err RecHandleMunge (DmOpenRef dbRef, UInt record,
                    Long offset, Long len,
                    const void*  pSrc, Long srcLen)
{
    VoidHand  hRec, hNew;
    Err              err = 0;
    UInt             attr;

    DmRecordInfo (dbRef, record, &attr, NULL, NULL);

    // if the record is currently not marked busy, busy it.
    if (dmRecAttrBusy & attr) {
        hRec = DmQueryRecord (dbRef, record);
    } else {
        hRec = DmGetRecord (dbRef, record);
    }
```

Record munge Routine Using MemUtilMunge

PalmSource 99

```
if (!hRec) {
    return dmErrIndexOutOfRange;
}

hNew = MemUtilMunge (hRec, offset, len, pSrc, srcLen,
&err, dbRef);
if (!err && hNew != hRec) {
    // handle changed, so reattach it
    err = DmAttachRecord (dbRef, &record, (Handle)hNew,
            (Handle*)&hRec);
    // we have to free memory, because MemUtilMunge
            won't if it gets reallocated
    if (hRec) {
            MemHandleFree (hRec);
    }
}
```

Record munge Routine Using MemUtilMunge

PalmSource 99

```
    if (!(dmRecAttrBusy & attr)) {
        // wasn't busy before, so unbusy it.
        DmReleaseRecord (dbRef, record, err == 0); // no
                change happened if there was an error
    }

    return err;
}
```

AppInfo munge Routine Using MemUtilMunge

PalmSource 99

```
Err AppInfoMunge (DmOpenRef dbRef,
                  Long      offset, Long size,
                  const void*      pSrc, Long srcLen)
{
    VoidHand  hInfo, hNew;
    Err               err = 0;
    LocalID           appInfoID;
    LocalID           dbID;
    UInt              cardNo;

    hInfo = AppInfoGetHandle (dbRef);
    if (hInfo == NULL) {
        return dmErrCantOpen;      // app info was not
        initialized or database is not open or corrupt
    }
```

AppInfo munge Routine Using MemUtilMunge

PalmSource 99

```
    hNew = MemUtilMunge (hInfo, offset, size, pSrc, srcLen,
    &err, dbRef);


    if (!err && hNew != hInfo) {
        // reallocated, so dispose of old one and reattach.
        // MemHandleFree (hInfo); orphaned by
                SetDatabaseInfo call
        appInfoID = MemHandleToLocalID (hNew);


        DmOpenDatabaseInfo(dbRef, &dbID, NULL, NULL,
                &cardNo, NULL);
        DmSetDatabaseInfo(cardNo, dbID, NULL, NULL, NULL,
                NULL, NULL, NULL, NULL, &appInfoID, NULL,
                NULL, NULL);
    }
    return err;
}
```



Problem-Solving Insights: Optimizing API Calls

Scott Johnson
Principal Technical Staff
Oracle Corporation



Why Optimize API Calls?

- Palm OS® software uses a trap dispatch system instead of static linking or dynamic linking
- Some APIs use a 2nd level of dispatching
  - Text Manager in device ROM
  - Text Manager in PalmOSGlue library
  - Future APIs they won't tell me about
- Sometimes the dispatch overhead hurts
  - But Palm OS software gives you a back door

A Simple Function to Optimize

Find width of the widest character in a string

```
SWord GetMaxWidth(Char *str, int len)
{
   SWord maxWidth = 0;
   for (int i = 0; i < len; i++)
   {
      Char ch = str[i];
      SWord w = FntCharWidth(ch);
      maxWidth = max(w, maxWidth);
   }
   return maxWidth;
}
```

Cache Function Address with SysGetTrapAddress

PalmSource 99

```
typedef SWord (*FntCharWidthPtr)(Char);

FntCharWidthPtr fntCharWidthP =
          (FntCharWidthPtr)
               SysGetTrapAddress(
                    sysTrapFntCharWidth);

SWord maxWidth = 0;
for (int i = 0; i < len; i++)
{
   Char ch = str[i];
   SWord w = fntCharWidthP(ch);
   maxWidth = max(w, maxWidth);
}
```

Upgrade To Text Manager: FntCharWidth → TxtCharWidth

PalmSource 99

```
typedef SWord (*TxtCharWidthPtr)(WChar);

TxtCharWidthPtr txtCharWidthP =
          (TxtCharWidthPtr)
               IntlGetRoutineAddress(
                    intlTxtCharWidth);

SWord maxWidth = 0;
for (int i = 0; i < len; i++)
{
   Char ch = str[i];
   SWord w = txtCharWidthP(ch);
   maxWidth = max(w, maxWidth);
}
```

Oops, Need TxtGetNextChar Too

PalmSource 99

```
typedef SWord (*TxtCharWidthPtr)(WChar);

TxtCharWidthPtr txtCharWidthP =
          (TxtCharWidthPtr)
               IntlGetRoutineAddress(
                    intlTxtCharWidth);

typedef Word (*TxtGetNextCharPtr)
          (const Char *, ULong, WChar *);

TxtGetNextCharPtr txtGetNextCharP =
          (TxtGetNextCharPtr)
               IntlGetRoutineAddress(
                    intlTxtGetNextChar);
```

...and the PalmOSGlue Library

PalmSource 99

```
typedef SWord (*TxtCharWidthPtr)(WChar);

TxtCharWidthPtr txtCharWidthP =
          (TxtCharWidthPtr)
               IntlGlueGetRoutineAddress(
                    TxtGlueCharWidth);

typedef Word (*TxtGetNextCharPtr)
          (const Char *, ULong, WChar *);

TxtGetNextCharPtr txtGetNextCharP =
          (TxtGetNextCharPtr)
               IntlGlueGetRoutineAddress(
                    TxtGlueGetNextChar);
```

What a Mess!

- Hard to write, read, and maintain
  - Needs one new **`typedef`** per API call
  - Needs one extra call per API to:
    - **`SysGetTrapAddress`**
    - **`IntlGetRoutineAddress`**
    - **`IntlGlueGetRoutineAddress`**
    - Or future functions of this kind
  - Needs one ugly pointer cast per API call
- Can this mess be cleaned up?
  - Yes! That's the point of this presentation

Let's Start All Over Again…


```
SWord GetMaxWidth(Char *str, int len)
{


   SWord maxWidth = 0;
   for (int i = 0; i < len; i++)
   {
      Char ch = str[i];
      SWord w =     FntCharWidth(ch);
      maxWidth = max(w, maxWidth);
   }
   return maxWidth;
}
```

SpeedTrap: Painless Pointers

PalmSource 99

```
SWord GetMaxWidth(Char *str, int len)
{
   SpeedTrap FCW(sysTrapFntCharWidth);

   SWord maxWidth = 0;
   for (int i = 0; i < len; i++)
   {
      Char ch = str[i];
      SWord w = FCW.FntCharWidth(ch);
      maxWidth = max(w, maxWidth);
   }
   return maxWidth;
}
```

How Does It Work? Look For the union Label

The old "union of function pointers" technique

```
union SpeedTrap
{
   SWord (*FntCharWidth)(Char);

   SWord (*TxtCharWidth)(WChar);

   Word  (*TxtGetNextChar)
         (const Char *, ULong, WChar *);

   /* … */

};
```

Overloaded Constructors for Versatile Usage

PalmSource 99

```
union SpeedTrap
{
   /* … */

   SpeedTrap(SysTrapNumber);

   SpeedTrap(IntlSelector);

   SpeedTrap(void *glueFunction);

   void *fn; // generic function ptr

   /* … */
};
```

Overloaded Constructors for Versatile Usage (Cont.)

PalmSource 99

```
SpeedTrap::SpeedTrap(SysTrapNumber trap)
:
   fn(SysGetTrapAddress(trap))
{}

SpeedTrap::SpeedTrap(IntlSelector sel)
:
   fn(IntlGetRoutineAddress(sel))
{}

SpeedTrap::SpeedTrap(void *glueFn)
:
   fn(IntlGlueGetRoutineAddress(glueFn))
{}
```

Complete Example Using the PalmOSGlue Library

PalmSource 99

```
SpeedTrap TGNC(TxtGlueGetNextChar);
SpeedTrap TCW(TxtGlueCharWidth);

SWord maxWidth = 0;
for (Word pos = 0; pos < len; )
{
   WChar ch;
   pos += TGNC.TxtGetNextChar(str,
                              pos,
                              &ch);

   SWord w = TCW.TxtCharWidth(ch);
   maxWidth = max(w, maxWidth);
}
```

Performance Considerations

PalmSource 99

- This technique does speed up some calls
  - Some base APIs which are very simple
  - Text Manager APIs in PalmOSGlue library
- Does not help for all system APIs
  - Complex APIs which call many other traps
- Be conservative using this technique
  - Measure the effectiveness
  - This example got about 2.5 times faster

Palm OS® Emulator Considerations

PalmSource 99

- Don’t use this for Memory Manager APIs
  - **`MemHandleNew`**, **`MemHandleLock`**, etc.
  - Causes harmless but annoying warnings
- If Emulator complains, back off
  - Never disable Emulator debug checks
- Benchmark on device, not Emulator
  - Emulator’s trap dispatch timing is not accurate

C Language Considerations

PalmSource 99

- SpeedTrap is easier to implement in C++
  - Constructor functions
  - Many other syntax advantages
- C language can be used too
  - Constructors change to regular functions
  - The rest of SpeedTrap stays the same
- Consider using C++ for these advantages
  - You don’t need to redesign your application
  - Just enable C++ and keep writing code

PalmSource™ 99

Problem-Solving Insights: Hunting Down Dynamic Memory Problems

Alan Pinstein
President, Synergy Solutions, Inc.

3Com
More connected.

Tools of the Trade

PalmSource 99

- POSE (Palm OS® Emulator)
  - Gremlins
  - Debug Options
- Palm OS Console
- Source Code
  - Consistent coding practices
  - Debugging asserts
  - Debugging routines

Using POSE to Track Down Problems

PalmSource 99

- Enable all of the ‘debug’ options in POSE
- Use Gremlins
  - 1,000,000 Gremlins will mercilessly destroy your application

Using the Palm OS® Console

PalmSource 99

- Console only work with OS 2.0.x or 3.0.x
- Exploring the heap
  - ht 0
- MDebug modes
  - mdebug -partial
- Inducing low memory conditions
  - hf 0 1024

Error Checking and Assert Examples

PalmSource 99

- Use high-level error checking routines in your code to catch problems when they happen

```
VoidPtr p = SynMemPtrNew(sz, “No memory in MyRoutine“);
if (!p)
    goto ErrExit;

VoidPtr SynMemPtrNew(ULong s, CharPtr msg)
{
    VoidPtr p;
    // assert for illegal request
    ErrFatalDisplayIf(s == 0, “0-length ptr requested.“);
    p = MemPtrNew(s);
    if ( !p )
        SynErrAlert(synErrNotEnoughSpace, msg);
    return p;
}
```


PalmSource™ 99
3Com
More connected.